

Übung: Daten transformieren und visualisieren

Am Beispiel der Tagesschau-Daten

Prof. Dr. Nicolas Meseth

In diesem Teil fokussieren wir uns auf die strukturierten Variablen des Tagesschau-Datensatzes. Wir lernen, welche Transformationen wir mit strukturierten Daten durchführen können und wie wir sie sinnvoll einsetzen, um explorative Fragen visuell zu beantworten. Dabei lernen wir verschiedene Visualisierungstechniken kennen, um die Daten ansprechend und informativ darzustellen.

Schritt 1: Projekt aufsetzen und Daten einlesen

Die Aufgaben 1 bis 4 sind für euch nur relevant, wenn ihr noch kein Projekt angelegt habt. Wenn ihr bereits ein Projekt für eine vorherige Übung erstellt habt, könnt ihr die Aufgaben 1 bis 3 sowie 5 auslassen und nur das neue R-Skript in Aufgabe 4 anlegen.

1. Erstellt ein neues Projekt für diese Übung und öffnet es in unserer Entwicklungsumgebung Positron. Stellt sicher, dass ihr eine *virtuelle R-Umgebung* für das Projekt erstellt und aktiviert habt.

2. Installiert die folgenden Pakete in eurer virtuellen Umgebung: `tidyverse`, `janitor`, `skimr` und `ggridges`. Verwendet das Metapaket `pacman`, damit ihr die Pakete mit einem einzigen Befehl installieren und laden könnt.

3. Ladet den Datensatz `tagesschau.zip` auf euren Computer herunter und speichert ihn in einem Unterordner `data/` in eurem Projektverzeichnis.

4. Erzeugt einen neuen Ordner `scripts/` in eurem Projektverzeichnis und erstellt eine neue R-Skriptdatei `transform_visualize_news.R` in diesem Ordner.

5. Der Datensatz liegt als komprimierte ZIP-Datei vor. Entpackt die Datei, das könnt ihr direkt in R mit `unzip()` erledigen. Schaut euch die entpackte Datei genauer an und prüft, wie das Format aussieht, zum Beispiel Trennzeichen und Zeichenkodierung. Lest die Daten dann mit einer geeigneten Funktion aus `readr` ein und speichert den Code in eurer Skriptdatei.

Variablen auswählen und Beobachtungen filtern und sortieren

Bevor ihr komplexere Analysen durchführt, lernt ihr in diesem Teil die grundlegenden `dplyr`-Verben `select()`, `filter()` und `arrange()` kennen. Diese drei Funktionen allein erlauben es euch schon ohne Visualisierung viele konkrete Fragen an einen Datensatz zu beantworten.

6. Der Datensatz enthält 21 Variablen, aber für viele Analysen braucht ihr davon nur einen kleinen Teil. Erstellt einen reduzierten Datensatz `news_slim`, der ausschließlich die Variablen `title`, `date_time`, `ressort`, `tag`, `author` und `word_count` enthält. Verwendet dafür `select()`. Wie viele Zeilen hat dieser reduzierte Datensatz?

7. Filtert den Datensatz `news_slim` auf alle Artikel aus dem Ressort “inland” und speichert das Ergebnis als `news_inland`. Wie viele Inland-Artikel enthält der Datensatz insgesamt?

8. Wie viele Artikel aus dem Ressort “ausland” wurden seit dem 1. Januar 2020 veröffentlicht *und* haben mehr als 800 Wörter? Von welchem Datum stammt der älteste dieser Artikel?

9. Die Variable `author` enthält für viele Artikel keine Angabe. Ermittelt zunächst, welchen Anteil in Prozent die Artikel ohne Autorengabe am Gesamtdatensatz ausmachen. Erstellt dann eine gefilterte Version `news_with_author`, die nur Zeilen *mit* Autorengabe enthält.

Was könnte es inhaltlich bedeuten, wenn kein Autor angegeben ist?

10. Welche zehn Artikel haben die meisten Wörter? Verwendet `arrange()` mit `desc()` und `head()`, oder alternativ `slice_max()`, um die zehn längsten Artikel zu identifizieren. Gebt `title`, `ressort` und `word_count` aus. In welchen Ressorts erscheinen die besonders langen Artikel?

Neue Variablen erzeugen

Mit `mutate()` könnt ihr bestehende Datensätze um neue, berechnete Variablen erweitern, ohne dabei andere Spalten oder die Zeilenanzahl zu verändern. In diesem Teil lernt ihr außerdem, wie ihr Datentypen anpasst, Datumsvariablen zerlegt und kategoriale Einteilungen vornehmt.

11. Die Variable `date_time` enthält sowohl Datum als auch Uhrzeit. Für viele Analysen brauchen wir sie in Einzelteilen. Fügt dem Datensatz `news_slim` mit `mutate()` drei neue Variablen hinzu:

- `year`, das Erscheinungsjahr
- `month`, den Erscheinungsmonat als Zahl von 1 bis 12
- `weekday`, den Wochentag als geordneter Faktor, Funktion: `wday(..., label = TRUE)`

Speichert den erweiterten Datensatz wieder als `news_slim`. Welche Wochentage sind im Datensatz vertreten, und wie heißen sie in R?

12. Die Variable `ressort` wurde beim Einlesen als `character` gespeichert, ist inhaltlich aber eine nominale kategoriale Variable. Wandelt sie mit `mutate()` und `as_factor()` in einen Faktor um. Überprüft mit `levels()`, welche Ausprägungen vorhanden sind, und mit `nlevels()`, wie viele es insgesamt gibt.

Macht dasselbe mit der Variable `tag`. Wie viele verschiedene Tags enthält der Datensatz? Fällt euch dabei etwas Unerwartetes auf?

13. Auch aus Textvariablen können wir nützliche Kenngrößen ableiten. Erstellt mit `mutate()` und Funktionen aus `stringr` zwei neue Variablen:

- `title_length`, die Zeichenanzahl des Titels
- `title_word_count`, die geschätzte Wortanzahl des Titels, zählt dafür die Leerzeichen und addiert 1

Berechnet für `title_length` Minimum, Median und Maximum. Was ist der kürzeste und was der längste Titel im Datensatz? Druckt die Artikel mit dem kürzesten und dem längsten Titel auf der Konsole aus.

14. Die Funktion `transmute()` verhält sich wie `mutate()`, behält aber im Ergebnis nur die neu erzeugten Spalten. Erstellt mit `transmute()` einen neuen kompakten Datensatz `news_timeline`, der ausschließlich folgende Variablen enthält: `year`, `month`, `weekday`, `ressort`, `word_count` sowie `title_length`, aus Aufgabe 13. Speichert diesen Datensatz

für spätere Analysen.

In welchen Situationen ist `transmute()` praktischer als `mutate()`? Nennt ein konkretes Beispiel.

15. Metrische Variablen lassen sich sinnvoll in inhaltliche Kategorien oder Gruppen einteilen. Erstellt mit `mutate()` und `case_when()` eine neue Variable `article_type`, die Artikel nach ihrer Länge einordnet:

- "Kurzmeldung", weniger als 200 Wörter
- "Standardartikel", 200 bis unter 700 Wörter
- "Langer Artikel", 700 Wörter und mehr

Wandelt `article_type` anschließend in einen Faktor mit einer inhaltlich sinnvollen Reihenfolge um. Berechnet, wie viele Artikel auf jede Kategorie entfallen, und ermittelt den Anteil in Prozent für jede Kategorie.

Daten gruppieren und zusammenfassen

`group_by()` und `summarize()` sind das mächtigste Werkzeugpaar in `dplyr`. Mit ihnen berechnet ihr beliebige Kenngrößen für beliebige Gruppen, von der einfachen Häufigkeitsauszählung bis hin zu differenzierten Gruppenstatistiken.

16. Berechnet die Anzahl der Artikel pro Ressort und sortiert das Ergebnis absteigend nach Häufigkeit. Welches ist das häufigste Ressort? Welche Ressorts enthalten weniger als 100 Artikel? Notiert euch die sechs häufigsten Ressorts für spätere Analysen. Wir wollen diese Gruppe als "Hauptressorts" bezeichnen.

17. Berechnet für die sechs Hauptressorts "inland", "ausland", "wirtschaft", "wissen", "investigativ" und "faktenfinder" folgende Kenngrößen in einem einzigen `summarize()`-Aufruf:

- Anzahl der Artikel, `n`
- Median der Wortanzahl, `median_word_count`
- mittlere Wortanzahl, gerundet auf ganze Zahlen, `mean_word_count`
- Anteil der Artikel mit namentlichem Autor in Prozent. Dabei gilt, ein Autor gilt als namentlich, wenn `author` weder `NA` noch "tagesschau.de" ist

Sortiert das Ergebnis absteigend nach der mittleren Wortanzahl. Welches Ressort produziert im Schnitt die längsten Artikel, und welches die kürzesten?

18. Wie hat sich die Artikelanzahl über die Jahre entwickelt? Berechnet für jedes Jahr die Gesamtanzahl aller Artikel sowie separat die Anzahl für die drei Ressorts "inland", "ausland" und "wirtschaft".

Fällt euch etwas an den Jahren 2006 und 2026 auf? Warum könnten diese Jahrgänge unvollständig sein, und wie solltet ihr das bei Interpretationen berücksichtigen?

19. `group_by()` funktioniert nicht nur mit `summarize()`, sondern auch mit `mutate()`. Der entscheidende Unterschied ist, `summarize()` verdichtet den Datensatz auf eine Zeile pro Gruppe, während `mutate()` alle Originalzeilen beibehält und lediglich eine neue Spalte hinzufügt.

Fügt dem Datensatz, gefiltert auf die Hauptressorts, mit `group_by()` und `mutate()` zwei neue Variablen hinzu:

- `ressort_median_wc`, der Median der Wortanzahl *des jeweiligen Ressorts*
- `above_median`, ein logischer Wert, `TRUE` oder `FALSE`, der angibt, ob der Artikel über dem Ressort-Median liegt

Wie viele Artikel im Ressort "inland" liegen über dem eigenen Ressort-Median? Ist das überraschend?

Häufigkeiten und Verteilungen visualisieren

Die grundlegenden Visualisierungsformen für einzelne Variablen sind das Balkendiagramm für kategoriale und das Histogramm für metrische Variablen. In diesem Teil lernt ihr, diese Formen gezielt einzusetzen und ansprechend zu gestalten.

20. Erstellt ein Balkendiagramm, das die Häufigkeitsverteilung der Ressorts zeigt. Filtert vorab auf die sechs Hauptressorts. Die Balken sollen nach Häufigkeit *absteigend* sortiert sein.

Beschriftet beide Achsen sinnvoll, wählt einen aussagekräftigen Diagrammtitel und verwendet ein schlichtes Theme.

21. Erstellt ein Histogramm der Variable `word_count`. Filtert zuvor Artikel mit mehr als 3.000 Wörtern heraus, damit die Verteilung übersichtlich bleibt. Experimentiert mit verschiedenen `bins`-Werten, zum Beispiel 30, 60 und 100, um eine aussagekräftige Klasseneinteilung zu finden.

Fügt dem Histogramm mit `geom_vline()` eine vertikale Linie für den Median hinzu. Beschreibt die Verteilung. Ist sie symmetrisch, links- oder rechtsschief? Was sagt das über

typische Artikellängen auf tagesschau.de aus?

22. Untersucht, welche Tags am häufigsten vergeben wurden. Filtert zunächst alle Tags heraus, die mindestens 200 Mal vorkommen, und entfernt NA-Werte.

Achtung: Die Tags "FAKTENFINDER" und "#FAKTENFINDER" bezeichnen dasselbe Konzept. Bereinigt das mit `str_remove_all()`, bevor ihr zählt.

Stellt die Top-Tags als sortiertes Balkendiagramm dar, absteigend nach Häufigkeit. Welche thematischen Schwerpunkte lassen sich aus der Tag-Verteilung ablesen?

Gruppenvergleiche visualisieren

Nicht nur einzelne Variablen, sondern Unterschiede zwischen Gruppen sind oft am aufschlussreichsten. Boxplots, kombinierte Boxplot-Punkte-Darstellungen und Ridgeline-Plots sind etwa dafür geeignet. Heatmaps helfen, zwei kategoriale Dimensionen gleichzeitig zu erkunden.

23. Erstellt einen Boxplot, der die Verteilung der Wortanzahl für die sechs Hauptressorts zeigt. Filtert Artikel mit mehr als 3.000 Wörtern heraus. Sortiert die Ressorts nach ihrem Median, Tipp: `fct_reorder(ressort, word_count, .fun = median)`, sodass das Ressort mit dem niedrigsten Median oben steht. Verwendet `coord_flip()` für eine horizontale Darstellung.

Welches Ressort hat die größte Streuung? Welches die kleinste?

24. Ergänzt den Boxplot aus Aufgabe 23 um überlagerte Datenpunkte. Setzt `alpha = 0.1` und `size = 0.5`, um Überlappungen zu reduzieren. Achtet auf die Reihenfolge der `geom_-`Aufrufe. Der Boxplot sollte *über* den Punkten liegen, nicht darunter.

Ab welcher Datenmenge gerät diese Darstellung an ihre Grenzen? Was ist der Vorteil dieser kombinierten Form gegenüber einem reinen Boxplot? Schaut euch mal die `geom_sina` aus dem `ggforce`-Paket an.

25. Installiert das Paket `ggridges` und erstellt mit `geom_density_ridges()` ein Ridgeline-Diagramm der Wortanzahl-Verteilung für alle Hauptressorts, die Filterung auf `word_count < 3000` bleibt bestehen. Füllt die Flächen mit `fill = ressort` und wählt eine ansprechende Farbpalette.

Beschreibt den Unterschied zwischen einem Boxplot und einem Ridgeline-Diagramm. Welche Informationen zeigt jede Darstellungsform, welche verbirgt sie? Wann würdet ihr

welche Form bevorzugen?

26. Erstellt eine Heatmap, die zeigt, wie viele Artikel an welchem Wochentag in welchem Hauptressort erschienen. Berechnet dafür zunächst die Häufigkeiten pro Wochentag und Ressort mit `group_by()` und `summarize()`. Visualisiert das Ergebnis mit `geom_tile()` und `scale_fill_viridis_c()`.

Zeigt die Wochentage auf der x-Achse, Montag bis Sonntag, die Ressorts auf der y-Achse. Was fällt euch beim Wochenendverhalten auf? Unterscheiden sich die Ressorts darin?

Zeitliche Entwicklungen visualisieren

Der Tagesschau-Datensatz erstreckt sich über fast zwei Jahrzehnte. Zeitreihenvisualisierungen helfen uns, Trends, Wachstum und saisonale Muster zu erkennen.

27. Erstellt ein Liniendiagramm, das die Gesamtanzahl der Artikel pro Jahr zeigt. Markiert die Datenpunkte zusätzlich mit `geom_point()`. Beschriftet die y-Achse sinnvoll und dreht die x-Achsenbeschriftungen, falls nötig.

Welche Jahre stechen besonders hervor? Versucht, den starken Anstieg ab 2023 inhaltlich zu erklären. Berücksichtigt dabei, dass 2006 und 2026 möglicherweise keine vollständigen Jahrgänge sind. Wie könnte man diesen Sachverhalt im Diagramm kenntlich machen?

28. Erstellt ein Liniendiagramm, das die Artikelanzahl pro Jahr *für die drei Ressorts "inland", "ausland" und "wirtschaft" gleichzeitig* als drei farbige Linien zeigt. Fügt eine Legende hinzu und wählt aussagekräftige Farben mit `scale_color_manual()` oder einer vorgefertigten Palette.

Welches Ressort ist in den letzten Jahren am stärksten gewachsen? Gibt es Phasen, in denen ein Ressort vorübergehend besonders aktiv war?

29. Erstellt ein gestapeltes Flächendiagramm, `geom_area()`, der Artikelanzahl pro Jahr für die sechs Hauptressorts. Wählt `position = "stack"`, also gestapelt. Vergleicht die Darstellung mit dem Liniendiagramm aus Aufgabe 28.

Was lässt sich aus der Gesamtfläche ablesen, was im Liniendiagramm nicht sichtbar war? Welche Darstellungsform ist besser geeignet, um den Anteil einzelner Ressorts am Gesamtvolumen zu zeigen?

30. Erstellt eine Heatmap, die zeigt, wie viele Artikel pro Jahr *und* Monat erschienen. Die x-Achse soll die Monate zeigen, Januar bis Dezember, die y-Achse die Jahre, mit dem neuesten Jahr oben, Tipp: Faktor mit umgekehrter Reihenfolge. Verwendet `geom_tile()` und `scale_fill_viridis_c(option = "plasma")`.

Lassen sich saisonale Muster erkennen? Ab welchem Jahr nimmt die Datendichte deutlich zu? Was könnte das über die Entstehungsgeschichte des Datensatzes verraten?

Zusammenhänge und mehrdimensionale Analysen

Scatter-Plots sind das klassische Werkzeug, um Zusammenhänge zwischen zwei metrischen Variablen zu erkunden. Wenn eine dritte Variable ins Spiel kommt, setzen wir Farbe oder Facettierung ein.

31. Gibt es einen Zusammenhang zwischen der Länge eines Titels `title_length` und der Länge des zugehörigen Artikels `word_count`? Zieht zunächst mit `slice_sample(n = 3000)` ein Zufallssample, um die Darstellung übersichtlich zu halten. Erstellt dann einen Scatter-Plot der beiden Variablen und fügt mit `geom_smooth(method = "lm")` eine lineare Trendlinie hinzu.

Beschreibt Stärke und Richtung des sichtbaren Zusammenhangs. Überprüft euren visuellen Eindruck mit `cor()`. Wie stark ist die Korrelation tatsächlich? Ist das Ergebnis überraschend, und warum oder warum nicht?

32. Erweitert den Scatter-Plot aus Aufgabe 31 um eine dritte Dimension. Färbt die Punkte nach `ressort` ein, nur die sechs Hauptressorts. Verwendet anschließend `facet_wrap(~ressort)`, um für jedes Ressort ein eigenes Teildiagramm zu erzeugen.

Unterscheidet sich der Zusammenhang zwischen Titellänge und Artikellänge zwischen den Ressorts?

33. Untersucht den Tagesrhythmus der Redaktion. Berechnet für jede Stunde des Tages, 0 bis 23 Uhr, die Anzahl der veröffentlichten Artikel. Visualisiert das Ergebnis als Flächen- oder Balkendiagramm.

Wann ist die Redaktion am produktivsten? Wann ist die Aktivität am niedrigsten? Gibt es Muster, die auf einen typischen Redaktionsalltag hindeuten?

Datensätze verknüpfen mit Joins

In der Praxis liegen Informationen selten in einer einzigen Tabelle vor. *Joins* ermöglichen es, Tabellen über gemeinsame Spalten zu verknüpfen. In diesem Teil erstellt ihr eine Metadaten-Tabelle und verknüpft sie auf verschiedene Weisen mit dem Hauptdatensatz.

34. Erstellt in R eine kleine Metadaten-Tabelle `ressort_meta` mit folgenden Spalten:

- `ressort`, der Ressort-Schlüssel, wie er im Datensatz vorkommt, zum Beispiel "inland"
- `ressort_label`, ein leserfreundlicher Anzeigename, zum Beispiel "Inland"
- `themenbereich`, eine übergeordnete thematische Kategorie, zum Beispiel "Politik", "Wirtschaft", "Gesellschaft" oder "Faktencheck"

Befüllt die Tabelle mit Einträgen für mindestens die acht häufigsten Ressorts. Lasst dabei bewusst einige seltene Ressorts *weg*, das wird in den nächsten Aufgaben relevant.

35. Verknüpft den, auf die Hauptvariablen reduzierten, Hauptdatensatz mit `ressort_meta` über einen `left_join()`, verbunden über die gemeinsame Spalte `ressort`. Was bedeutet ein `left_join()` in diesem Kontext, welche Zeilen bleiben garantiert erhalten?

Prüft das Ergebnis. Für wie viele Artikel wurde kein passender Eintrag in `ressort_meta` gefunden, erkennbar an NA in `ressort_label`? Welche Ressorts sind das?

Erstellt anschließend ein Balkendiagramm, das die Anzahl der Artikel pro `themenbereich` zeigt, sortiert nach Häufigkeit. Behandelt Artikel ohne Match als eigene Kategorie "Nicht klassifiziert", Tipp: `replace_na()`, und hebt diese Kategorie farblich ab.

36. Führt denselben Join als `inner_join()` durch. Was ist der Unterschied zum `left_join()` aus Aufgabe 35? Wie viele Zeilen hat das Ergebnis im Vergleich?

Erstellt mit dem Ergebnis des `inner_join()` einen horizontalen Boxplot der Wortanzahl, gruppiert nach `themenbereich`, gefiltert auf `word_count < 3000`, sortiert nach Median. Welcher Themenbereich produziert die längsten Artikel? Und spiegelt das die inhaltliche Natur der zugehörigen Ressorts wider?

37. Verwendet `anti_join()`, um herauszufinden, welche Artikel im Hauptdatensatz einen Ressort-Wert haben, der *nicht* in `ressort_meta` vorkommt. Gebt die fehlenden Ressorts mit ihrer Häufigkeit aus, sortiert nach Häufigkeit.

Was ist der praktische Nutzen von `anti_join()` beim Arbeiten mit Lookup-Tabellen? Wie hängt der `anti_join()` mit dem `left_join()` zusammen? Was ist der Bezug

zwischen den Zeilen, die beim `left_join()` den Wert `NA` erhalten, und dem Ergebnis des `anti_join()`?

Erstellt ein Balkendiagramm der nicht gematchten Ressorts, nur solche mit mindestens 5 Artikeln, ohne `NA`. Hebt mit einer abweichenden Farbe die Ressorts hervor, die ihr für eine vollständigere Lookup-Tabelle als prioritär erachtet, zum Beispiel weil sie mehr als 100 Artikel enthalten. Was müsste in `ressort_meta` ergänzt werden, damit der `inner_join()` diese Artikel nicht mehr verliert?

Reflexion

38. Erklärt in eigenen Worten den Unterschied zwischen `summarize()` und `mutate()` in Kombination mit `group_by()`. Nennt je ein konkretes Beispiel aus dieser Übung, bei dem ihr lieber `summarize()` und eines, bei dem ihr lieber `group_by()` + `mutate()` eingesetzt habt und begründet eure Wahl.

39. Ihr habt in dieser Übung fast zwei Jahrzehnte Tagesschau-Berichterstattung analysiert. Welche drei Beobachtungen haben euch am meisten überrascht oder sind euch besonders aufgefallen? Versucht, jede Beobachtung auch inhaltlich, über das bloße Zahlenmuster hinaus, zu erklären oder einzuordnen.

40. Betrachtet alle Visualisierungsformen aus dieser Übung, Histogramm, Balkendiagramm, Boxplot, Boxplot mit Punktwolke, Ridgeline-Plot, Heatmap, Liniendiagramm, Flächendiagramm und Scatter-Plot. Erstellt eine eigene gedankliche Klassifikation. Welche Form ist für welche Kombination aus Fragestellung, univariat, bivariat, trivariat, und Variablentyp, metrisch, kategorial, zeitlich, am besten geeignet? Gibt es Überschneidungen, also Situationen, in denen mehrere Formen in Frage kämen? Und was bestimmt dann die Wahl?

41. Datenqualität ist ein unterschätztes, aber entscheidendes Thema. Nennt mindestens vier konkrete Hinweise auf potenzielle Qualitätsprobleme oder Einschränkungen im Tagesschau-Datensatz, die euch im Laufe dieser Übung aufgefallen sind. Denkt dabei an fehlende Werte, uneinheitliche Kodierungen, unvollständige Jahrgänge, Veränderungen der Datenquelle über die Zeit und mögliche Selektionseffekte. Wie würdet ihr diese Probleme in einer veröffentlichungswürdigen Analyse dokumentieren und behandeln?

42. In dieser Übung haben wir ausschließlich mit den strukturierten Merkmalen im Datensatz gearbeitet. Wie könnten wir die unstrukturierten Merkmale wie den Artikeltext oder die Titel in unsere Analysen einbeziehen? Was müssten wir vorher tun?